

Informe Técnico de Ingeniería de Datos

Integración de Datos, Procesos ETL y Visualización Analítica
mediante Dash y Docker

NETFLIX

Alumnos:

1. Bastian Roman
2. Ignacio Gomez
3. Vicente Contreras

Asignatura:

- Programación para la ciencia de datos

Docente: Victor Andres Trigo Rojas

Sección: 001D

1. RESUMEN EJECUTIVO

El presente informe describe el desarrollo e implementación del proyecto denominado *Netflix Analytics Dashboard*, una solución de análisis de datos orientada a demostrar competencias fundamentales en Ingeniería de Datos mediante la integración de múltiples fuentes de información, la aplicación de procesos ETL (Extract, Transform, Load), la construcción de visualizaciones interactivas y el despliegue de aplicaciones utilizando tecnologías de contenerización.

La solución fue desarrollada utilizando Python como lenguaje principal e integró información proveniente de tres fuentes distintas: un archivo CSV correspondiente al catálogo de Netflix, una base de datos SQLite que simuló métricas de consumo de contenido y una API externa denominada TVMaze, utilizada para enriquecer los registros con información adicional relacionada con programas de televisión.

Durante el desarrollo se implementó un proceso ETL completo encargado de extraer la información desde las distintas fuentes, transformarla mediante procesos de limpieza e integración y cargar el resultado en una base de datos consolidada destinada al análisis y visualización. Posteriormente, la información procesada fue presentada mediante un dashboard interactivo construido con Dash y Plotly.

Con el propósito de facilitar la portabilidad y reproducibilidad de la solución, el proyecto fue contenerizado utilizando Docker y Docker Compose, permitiendo su ejecución en distintos entornos sin necesidad de configuraciones complejas.

El proyecto permitió demostrar de manera práctica conceptos relacionados con integración de datos, calidad de datos, procesamiento ETL, visualización analítica, documentación técnica y despliegue de aplicaciones modernas, constituyendo una implementación representativa de un flujo de trabajo típico dentro del ámbito de la Ingeniería de Datos.

2. INTRODUCCIÓN

La transformación digital ha provocado un crecimiento exponencial en la generación y almacenamiento de datos. Las organizaciones modernas dependen cada vez más de la capacidad de recopilar, integrar y analizar información proveniente de múltiples fuentes con el objetivo de obtener conocimiento útil para la toma de decisiones.

En este contexto, la Ingeniería de Datos desempeña un papel fundamental al proporcionar mecanismos que permiten extraer información desde diferentes sistemas, transformarla para garantizar su calidad y disponibilidad, y finalmente ponerla a disposición de usuarios y aplicaciones analíticas.

El proyecto Netflix Analytics Dashboard fue desarrollado con el propósito de demostrar la aplicación práctica de estos conceptos mediante la construcción de una solución integral

capaz de integrar datos provenientes de diversas fuentes, procesarlos mediante un pipeline ETL y visualizarlos a través de una interfaz gráfica interactiva.

El proyecto fue desarrollado en el contexto de una evaluación académica de Ingeniería de Datos, cuyo propósito consistió en demostrar competencias relacionadas con la integración de datos, diseño de procesos ETL, visualización de información, documentación técnica y despliegue de soluciones analíticas utilizando tecnologías modernas.

Para lograr este objetivo se utilizaron tres fuentes de información complementarias. La primera correspondió a un conjunto de datos en formato CSV que contenía información histórica del catálogo de Netflix. La segunda fue una base de datos SQLite que simuló métricas de visualización de contenido. Finalmente, se incorporó una API externa denominada TVMaze para enriquecer los registros mediante datos adicionales no disponibles en las fuentes originales.

Posteriormente, toda la información fue integrada mediante un proceso ETL desarrollado en Python y almacenada en una base de datos consolidada utilizada como fuente para un dashboard analítico construido con Dash y Plotly. Finalmente, la solución fue contenerizada utilizando Docker y Docker Compose, permitiendo una implementación portable y fácilmente reproducible.

3. OBJETIVOS

3.1 Objetivo General

Construir un pipeline de datos capaz de integrar información desde distintas fuentes, transformarla y presentarla mediante un dashboard interactivo para apoyar la toma de decisiones.

3.2 Objetivos Específicos

- A. Integrar información proveniente de un archivo CSV, una base de datos SQLite y una API REST externa.
- B. Diseñar e implementar un proceso ETL para consolidar la información proveniente de múltiples fuentes.
- C. Aplicar procesos de limpieza, validación y transformación de datos.
- D. Construir visualizaciones interactivas orientadas al análisis de información.
- E. Implementar una solución contenerizada mediante Docker y Docker Compose.
- F. Generar documentación técnica que facilite la comprensión, mantenimiento y despliegue del sistema.

4. DESCRIPCIÓN DEL PROBLEMA

Las organizaciones suelen almacenar información en múltiples sistemas y formatos, lo que dificulta la obtención de una visión integrada de los datos. Esta situación genera problemas relacionados con la dispersión de la información, duplicidad de registros, diferencias de formato y limitaciones para realizar análisis eficientes.

En el contexto del presente proyecto, la información relevante se encontraba distribuida entre distintas fuentes de datos. Por una parte, existía un catálogo histórico de contenido disponible en formato CSV. Por otra, se disponía de una base de datos que simulaba métricas de visualización asociadas al consumo de contenido. Adicionalmente, existía información complementaria disponible mediante servicios externos accesibles a través de APIs.

La principal problemática consistió en integrar estas fuentes heterogéneas dentro de una estructura unificada que permitiera analizar la información de manera eficiente y presentar resultados mediante herramientas visuales accesibles para los usuarios.

Para resolver este desafío se diseñó una arquitectura basada en procesos ETL capaces de consolidar la información proveniente de diferentes orígenes, garantizando consistencia, calidad y disponibilidad para su posterior análisis.

La solución desarrollada permitió centralizar la información en una única fuente de datos, enriquecer los registros mediante información externa y proporcionar una plataforma visual para explorar tendencias relacionadas con películas, series, países de origen, géneros y otros atributos relevantes del catálogo analizado.

5. ARQUITECTURA DE LA SOLUCIÓN

La arquitectura implementada se basó en un enfoque modular orientado a separar responsabilidades y facilitar el mantenimiento del proyecto. Cada componente fue diseñado para cumplir una función específica dentro del flujo de procesamiento de datos.

La solución se estructuró mediante varias carpetas especializadas. La carpeta **data** almacenó los datasets originales y las bases de datos utilizadas durante el proyecto. La carpeta **etl** concentró la lógica encargada de la extracción, transformación y carga de información. La carpeta **dashboards** contuvo la aplicación desarrollada con Dash y Plotly para la visualización interactiva de datos. Finalmente, la carpeta **docs** reunió toda la documentación técnica asociada al proyecto.

El flujo general de procesamiento comenzó con la extracción de información desde el archivo CSV de Netflix, la base de datos SQLite y la API TVMaze. Posteriormente, los datos fueron transformados mediante procesos de limpieza, integración y enriquecimiento. Finalmente, la información consolidada fue almacenada en una base de datos SQLite

denominada `dataset_final_dashboard.db`, utilizada como fuente principal para el dashboard analítico.

La arquitectura implementada permitió mantener una separación clara entre las fuentes de datos, los procesos de transformación y la capa de visualización, favoreciendo la escalabilidad y mantenibilidad de la solución.

Tabla 1. Componentes principales de la arquitectura

Componente	Función
CSV Netflix	Fuente de datos histórica
SQLite	Simulación de métricas de negocio
TVMaze API	Fuente externa de enriquecimiento
ETL Python	Integración y transformación de datos
Dataset Final SQLite	Almacenamiento consolidado
Dash	Interfaz analítica
Plotly	Visualizaciones interactivas
Docker	Contenerización
Docker Compose	Automatización del despliegue
Modelo ML (Scikit-learn)	Predicción del tipo de contenido (Movie / TV Show)
API Flask	Exposición del modelo de ML mediante servicio REST
Transformaciones Avanzadas	Pivot, reshape, chunking y broadcasting sobre el catálogo completo

El flujo de datos implementado puede representarse conceptualmente de la siguiente forma:

CSV Netflix + SQLite + TVMaze API → ETL → Base Consolidada → Dashboard → Docker/Docker Compose

6. DESCRIPCIÓN DE LAS FUENTES DE DATOS

Uno de los aspectos fundamentales del proyecto consistió en la integración de información proveniente de múltiples fuentes de datos. Esta estrategia permitió demostrar de manera práctica uno de los principios centrales de la Ingeniería de Datos: la consolidación de información heterogénea para generar valor analítico.

La solución desarrollada utilizó tres fuentes de datos distintas, cada una con características y propósitos específicos dentro del proceso ETL.

6.1 Archivo CSV del Catálogo Netflix

La primera fuente de información correspondió al archivo denominado *netflix_titles.csv*, el cual contiene información histórica sobre películas y series disponibles dentro del catálogo de Netflix.

Este conjunto de datos proporcionó los atributos descriptivos principales utilizados durante el análisis, incluyendo información relacionada con títulos, tipos de contenido, países de origen, géneros, directores, clasificaciones y años de lanzamiento.

Tabla 2. Información principal obtenida desde el CSV

Campo	Descripción
show_id	Identificador único del contenido
title	Nombre del contenido
type	Tipo de contenido
director	Director asociado
cast	Reparto principal
country	País de origen
date_added	Fecha de incorporación
release_year	Año de lanzamiento
rating	Clasificación de audiencia

duration	Duración
listed_in	Géneros
description	Descripción del contenido

Esta fuente constituyó la base principal sobre la cual se desarrolló el resto del proceso de integración.

6.2 Base de Datos SQLite

La segunda fuente correspondió a una base de datos SQLite denominada *catalogo_netflix.db*.

Dentro de esta base se almacenó la tabla *historial_visualizaciones*, la cual fue creada con el objetivo de simular información de negocio relacionada con el consumo de contenido por parte de los usuarios.

El uso de SQLite permitió incorporar una fuente estructurada similar a las utilizadas en entornos empresariales, facilitando la aplicación de consultas SQL y procesos de integración de datos.

Tabla 3. Información contenida en SQLite

Campo	Descripción
title	Nombre del contenido
type	Tipo de contenido
visualizaciones	Cantidad simulada de visualizaciones

La utilización de SQLite resultó adecuada debido a su simplicidad, portabilidad y facilidad de integración con Python.

6.3 API Externa TVMaze

La tercera fuente utilizada fue la API pública TVMaze.

Esta API fue incorporada con el propósito de enriquecer los registros existentes mediante información externa obtenida dinámicamente desde Internet.

A través de consultas basadas en el nombre de cada título, fue posible recuperar atributos adicionales relacionados con programas de televisión y series.

Tabla 4. Datos incorporados desde TVMaze

Campo	Descripción
tmvaze_rating	Evaluación del contenido
tmvaze_language	Idioma principal
tmvaze_status	Estado de emisión

La incorporación de esta API permitió demostrar la capacidad de integrar servicios externos dentro de un pipeline ETL, una práctica ampliamente utilizada en proyectos reales de Ingeniería de Datos.

7. DISEÑO E IMPLEMENTACIÓN DEL ETL

El proceso ETL constituyó el núcleo de la solución desarrollada. Su función principal consistió en extraer información desde distintas fuentes, transformarla para garantizar su calidad y consistencia, y finalmente cargarla en una estructura consolidada destinada al análisis.

La implementación fue realizada utilizando Python y la biblioteca Pandas, permitiendo manipular grandes cantidades de información de manera eficiente.

7.1 Fase Extract

Durante esta etapa se realizó la obtención de información desde las tres fuentes de datos disponibles.

En primer lugar, se cargó el archivo CSV correspondiente al catálogo Netflix. Posteriormente, se estableció una conexión con la base de datos SQLite para recuperar las métricas de visualización simuladas. Finalmente, se ejecutaron consultas HTTP hacia la API TVMaze para obtener información complementaria asociada a los títulos presentes en el catálogo.

Esta fase permitió reunir información distribuida en diferentes formatos y tecnologías.

7.2 Fase Transform

La etapa de transformación tuvo como objetivo preparar los datos para su posterior análisis.

Las principales operaciones realizadas fueron:

- A. Integración de las distintas fuentes mediante el campo title.
- B. Eliminación de columnas duplicadas generadas por los procesos de unión.
- C. Tratamiento de valores nulos mediante técnicas de sustitución.
- D. Normalización de atributos textuales.
- E. Incorporación de información proveniente de TVMaze.
- F. Consolidación de todos los registros en una única estructura tabular.

Estas transformaciones permitieron obtener un conjunto de datos consistente y preparado para ser consumido por el dashboard.

7.3 Fase Load

Una vez finalizadas las transformaciones, los datos fueron almacenados en una nueva base de datos SQLite denominada:

`dataset_final_dashboard.db`

Dentro de esta base se generó la tabla:

`vista_dashboard`

Esta tabla consolidó toda la información necesaria para alimentar las visualizaciones del dashboard sin necesidad de volver a consultar las fuentes originales.

La utilización de una base de datos consolidada permitió mejorar el rendimiento y simplificar la arquitectura de visualización.

7.4 Valor Académico del ETL

La implementación del proceso ETL permitió demostrar de forma práctica las tres etapas fundamentales de la Ingeniería de Datos.

La fase Extract evidenció la capacidad de conectarse a múltiples fuentes heterogéneas.

La fase Transform demostró la aplicación de procesos de limpieza, integración y enriquecimiento de información.

Finalmente, la fase Load permitió almacenar los resultados en una estructura optimizada para el análisis y la generación de visualizaciones.

7.5 Transformaciones Avanzadas de Datos

Complementando el pipeline ETL principal, se desarrolló el módulo `analisis_avanzado.py`, que demuestra de forma explícita cuatro técnicas de transformación avanzada sobre el catálogo completo de Netflix (8.807 registros): `pivot`, `reshape`, procesamiento por chunks y vectorización mediante `broadcasting`.

En primer lugar, se construyó una tabla pivote (`pivot_table`) que cruza la década de lanzamiento con la clasificación de audiencia, contando la cantidad de títulos en cada combinación y agregando totales de fila y columna. Un extracto de esta tabla se muestra a continuación:

Tabla 10. Extracto de la tabla pivote (década × clasificación de audiencia)

Década	TV-14	TV-MA	PG-13	Total
2000	142	139	118	612
2010	612	1310	176	3229
2020	298	1064	62	2450

Posteriormente, esa misma tabla se transformó de formato ancho a formato largo mediante la técnica de `reshape` (`pd.melt`), obteniendo una fila por cada combinación década-clasificación con su respectivo conteo. Este formato es el que habitualmente requieren las librerías de visualización, como `Plotly Express`, para construir gráficos de forma directa.

En tercer lugar, se implementó procesamiento por chunks (lotes), leyendo el archivo CSV en bloques de 1.000 filas en lugar de cargarlo completo en memoria. El catálogo actual se procesó en 9 lotes, acumulando 8.807 filas en aproximadamente 0.11 segundos, con conteos de país acumulados de forma incremental entre lotes. Aunque el volumen actual cabe cómodamente en memoria, esta es la técnica correcta para escalar a archivos de millones de registros sin agotar los recursos disponibles.

Finalmente, se calculó un score de popularidad normalizado utilizando vectorización y `broadcasting` de `NumPy`: la media y la desviación estándar de las visualizaciones — ambos valores escalares — se restan y dividen directamente sobre el arreglo completo en una sola operación, sin recurrir a ningún ciclo explícito en Python. El resultado se reescaló a un rango de 0 a 100 para facilitar su interpretación.

Estas cuatro técnicas complementan la manipulación básica de datos (filtros, agrupaciones y uniones) realizada en el ETL principal, evidenciando un manejo más profundo de `Pandas` y `NumPy` orientado a la eficiencia y a la escalabilidad.

8. INTEGRACIÓN CON API EXTERNA

Uno de los elementos más relevantes del proyecto fue la incorporación de una API externa como fuente complementaria de información.

La API seleccionada fue TVMaze, un servicio público que proporciona información sobre series y programas de televisión mediante endpoints accesibles a través de solicitudes HTTP.

La integración fue implementada utilizando la biblioteca Requests de Python.

El proceso consistió en consultar la API utilizando el nombre de cada título disponible en el conjunto de datos. Posteriormente, la respuesta fue procesada en formato JSON para extraer únicamente los atributos relevantes para el proyecto.

Esta estrategia permitió enriquecer la información original sin modificar las fuentes de datos iniciales.

La incorporación de TVMaze aportó tres beneficios principales:

En primer lugar, permitió complementar la información existente con atributos no presentes en el catálogo Netflix.

En segundo lugar, facilitó la demostración práctica de integración de servicios externos dentro de un proceso ETL.

Finalmente, permitió construir visualizaciones adicionales dentro del dashboard relacionadas con idiomas, estados de emisión y valoraciones del contenido.

La utilización de APIs constituye una práctica habitual dentro de proyectos empresariales modernos, ya que permite enriquecer datos internos mediante fuentes externas especializadas.

8.1 Modelo de Machine Learning y API de Predicción

Además de la integración de las tres fuentes de datos, el proyecto incorpora un modelo de Machine Learning supervisado orientado a predecir el tipo de contenido (Movie o TV Show) a partir de sus metadatos, junto con una API REST propia que expone dicho modelo para su consumo por parte de otros sistemas.

El modelo se entrenó sobre el catálogo completo de Netflix (8.807 registros), a diferencia del resto del pipeline que trabaja sobre la muestra simulada de 30 títulos, con el fin de contar

con un volumen de datos suficiente para un entrenamiento y una validación estadísticamente confiables.

Las variables utilizadas para la predicción fueron: año de lanzamiento, década de lanzamiento, clasificación de audiencia (rating), país principal, cantidad de actores en el elenco, presencia de director registrado, largo de la descripción y mes de incorporación al catálogo. Se excluyeron deliberadamente los campos `listed_in` (género) y `duration`, ya que ambos delatan el tipo de contenido de forma casi directa y provocarían fuga de información (data leakage), invalidando el aprendizaje real del modelo.

Se entrenaron y compararon tres algoritmos de clasificación: Regresión Logística (modelo lineal, utilizado como referencia interpretable), Random Forest, con búsqueda de hiperparámetros mediante GridSearchCV, y Gradient Boosting. El preprocesamiento (escalado de variables numéricas y codificación one-hot de variables categóricas) se encapsuló junto con el modelo en un mismo pipeline de Scikit-learn, guardado como un único artefacto reutilizable.

Tabla 8. Comparación de modelos sobre el conjunto de prueba (20% de los datos, split estratificado)

Modelo	Accuracy	F1-score	ROC-AUC
Gradient Boosting (seleccionado)	0.957	0.930	0.981
Regresión Logística	0.957	0.929	0.978
Random Forest (tuned)	0.956	0.928	0.980

El modelo Gradient Boosting obtuvo el mejor desempeño y fue seleccionado como modelo final, con una exactitud (accuracy) de 95,7% y un F1-score de 0,930 para la clase minoritaria (TV Show) sobre datos nunca vistos durante el entrenamiento.

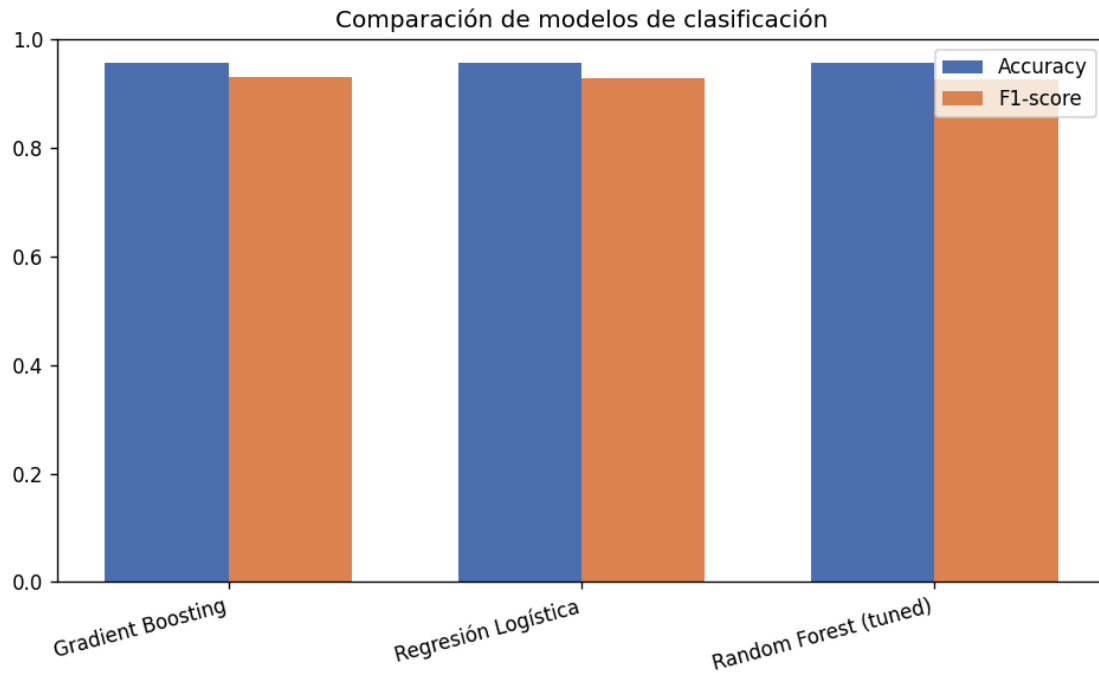


Figura 1. Comparación de accuracy y F1-score entre los tres modelos entrenados.

Un hallazgo relevante del análisis de interpretabilidad es que la variable más predictiva resultó ser la presencia de director registrado: 97% de las películas del catálogo tienen un director asociado, frente a solo un 9% de las series de TV. Este patrón refleja una convención real de cómo Netflix cataloga sus contenidos (las series rara vez se atribuyen a un único director) y no constituye un error de los datos. Como trabajo futuro, se identificó la oportunidad de reentrenar el modelo excluyendo esta variable, para evaluar la capacidad predictiva del resto de los atributos de forma más exigente.

El modelo entrenado se expone a través de una API REST construida con Flask, que corre como un servicio independiente dentro de la arquitectura Docker (puerto 5001, separado del dashboard en el puerto 8050). La API valida que las variables requeridas estén presentes antes de predecir, y deriva automáticamente la década de lanzamiento a partir del año si no se especifica.

Tabla 9. Endpoints disponibles en la API del modelo

Método	Endpoint	Descripción
GET	/health	Verifica que el servicio esté activo
GET	/model-info	Metadatos, features y métricas del modelo
GET	/predict/ejemplo	Predicción de demostración sin enviar datos
POST	/predict	Predicción para un título nuevo enviado en el body

9. DISEÑO DEL DASHBOARD

La visualización de información fue desarrollada mediante Dash y Plotly, dos tecnologías ampliamente utilizadas para la construcción de aplicaciones analíticas interactivas.

El objetivo principal del dashboard consistió en transformar datos complejos en representaciones gráficas comprensibles para los usuarios.

La interfaz fue diseñada siguiendo criterios de simplicidad, claridad visual y facilidad de interpretación.

9.1 Indicadores KPI

Se implementaron cuatro indicadores principales:

1. Total de títulos.
2. Total de películas.
3. Total de series.
4. Total de visualizaciones.

Estos indicadores proporcionan una visión rápida del volumen general de información disponible.

9.2 Distribución de Contenido

La visualización de distribución permite comparar la proporción existente entre películas y series presentes en el conjunto de datos.

Esta información resulta útil para comprender la composición general del catálogo.

9.3 Países con Mayor Presencia

El gráfico de países permite identificar cuáles son las naciones que poseen una mayor representación dentro del conjunto analizado.

Este análisis facilita la identificación de tendencias geográficas en la producción de contenido.

9.4 Géneros Predominantes

La visualización de géneros permite identificar las categorías más frecuentes dentro del catálogo.

Este tipo de análisis resulta útil para comprender la orientación general del contenido disponible.

9.5 Títulos Más Visualizados

La visualización de títulos más vistos permite identificar aquellos contenidos que presentan una mayor cantidad de visualizaciones simuladas.

Esta información puede ser utilizada como apoyo para procesos de análisis de popularidad o consumo.

9.6 Información Enriquecida desde TVMaze

Las visualizaciones asociadas a ratings e idiomas permitieron demostrar el valor agregado generado por la integración de la API externa.

Estas representaciones muestran información que originalmente no estaba presente en el dataset base.

10. CONTENERIZACIÓN CON DOCKER

Con el objetivo de garantizar portabilidad y facilidad de despliegue, la solución fue contenerizada utilizando Docker.

La contenerización permite encapsular la aplicación junto con todas sus dependencias dentro de un entorno aislado y reproducible.

Para ello se implementó un Dockerfile encargado de definir el entorno de ejecución del proyecto.

Entre las tareas realizadas por este archivo se encuentran:

1. Selección de la imagen base de Python.
2. Instalación de dependencias.
3. Copia del código fuente.

4. Configuración del puerto utilizado por Dash.
5. Definición del comando de inicio de la aplicación.

Adicionalmente, se implementó un archivo docker-compose.yml destinado a simplificar el despliegue del sistema.

La utilización de Docker Compose permitió automatizar la construcción y ejecución de la aplicación mediante un único comando:

```
docker compose up
```

Esta estrategia facilita la reproducibilidad del entorno y reduce significativamente los problemas asociados a diferencias entre sistemas operativos o configuraciones locales.

11. DOCUMENTACIÓN TÉCNICA GENERADA

La documentación constituye un elemento fundamental dentro del ciclo de vida de cualquier proyecto de software o datos.

Con el objetivo de facilitar la comprensión y mantenimiento de la solución, se generó un conjunto de documentos técnicos complementarios.

Tabla 5. Documentación desarrollada

Documento	Finalidad
-----------	-----------

README.md	Descripción general del proyecto
arquitectura.md	Explicación de la arquitectura implementada
api_tvmaze.md	Documentación de integración con TVMaze
manual_usuario.md	Instrucciones de instalación y uso

La existencia de esta documentación mejora la mantenibilidad del proyecto, facilita la incorporación de nuevos desarrolladores y permite comprender rápidamente la arquitectura y funcionamiento general de la solución.

Asimismo, constituye una evidencia importante del proceso de ingeniería aplicado durante el desarrollo.

12. RESULTADOS OBTENIDOS

La implementación del proyecto permitió demostrar exitosamente la integración de múltiples fuentes de datos mediante un proceso ETL completo, consolidando información heterogénea en una estructura única destinada al análisis y visualización.

Como resultado del proceso desarrollado, se obtuvo una base de datos consolidada denominada *dataset_final_dashboard.db*, la cual fue utilizada como fuente principal para alimentar el dashboard analítico.

Desde una perspectiva técnica, el proyecto logró integrar tres fuentes de información diferentes:

1. Un archivo CSV correspondiente al catálogo de Netflix.
2. Una base de datos SQLite que simuló métricas de visualización.
3. Una API externa utilizada para enriquecer los registros mediante información adicional.

La integración exitosa de estas fuentes permitió validar el correcto funcionamiento del pipeline ETL implementado.

Desde la perspectiva analítica, el dashboard desarrollado permitió explorar distintos aspectos del conjunto de datos. Entre los hallazgos observados se identificó la coexistencia de películas y series dentro del catálogo, la presencia de contenido asociado a múltiples países, la existencia de diversos géneros y la disponibilidad de información enriquecida proveniente de TVMaze.

Asimismo, las visualizaciones relacionadas con idiomas y ratings demostraron el valor agregado generado por la integración de información externa, permitiendo ampliar las capacidades analíticas de la solución.

Es importante señalar que el objetivo principal del proyecto no consistió en obtener conclusiones de negocio específicas, sino en demostrar la correcta implementación de procesos de Ingeniería de Datos orientados a la integración, transformación y visualización de información.

Tabla 6. Resultados alcanzados

Componente	Resultado
CSV Netflix	Integrado correctamente
SQLite	Integrado correctamente
TVMaze API	Integrada correctamente
ETL	Implementado y operativo
Dashboard	Implementado y operativo
Docker	Implementado y operativo
Docker Compose	Implementado y operativo
Documentación	Generada correctamente

Los resultados obtenidos evidenciaron el cumplimiento de los objetivos definidos para el proyecto y permitieron demostrar competencias prácticas relacionadas con Ingeniería de Datos, Business Intelligence y despliegue de aplicaciones analíticas.

13. BENEFICIOS DE LA SOLUCIÓN

La solución desarrollada presentó múltiples beneficios tanto desde una perspectiva académica como técnica.

En primer lugar, permitió demostrar la capacidad de integrar información proveniente de diferentes fuentes y formatos. Esta característica representa una de las tareas más comunes dentro de proyectos reales de Ingeniería de Datos.

En segundo lugar, la implementación de un proceso ETL facilitó la transformación y consolidación de datos, garantizando que la información utilizada por el dashboard se encontrara organizada y preparada para su análisis.

Otro beneficio relevante fue la incorporación de una API externa como mecanismo de enriquecimiento de datos. Esta práctica permitió ampliar el valor analítico de la información disponible sin necesidad de modificar las fuentes originales.

Desde la perspectiva de visualización, el dashboard desarrollado proporcionó una interfaz intuitiva capaz de representar información compleja mediante gráficos e indicadores fácilmente interpretables.

La contenerización mediante Docker constituyó otro beneficio importante, ya que permitió encapsular toda la solución dentro de un entorno reproducible y portable.

Finalmente, la documentación generada contribuyó significativamente a la mantenibilidad del proyecto, facilitando su comprensión, despliegue y posible evolución futura.

Tabla 7. Beneficios identificados

Beneficio	Impacto
Integración de múltiples fuentes	Consolidación de información
ETL automatizado	Mejora de calidad de datos
API externa	Enriquecimiento de registros
Dashboard interactivo	Mejor comprensión de datos
Docker	Portabilidad y despliegue
Documentación	Mantenibilidad del proyecto
Modelo de Machine Learning + API	Predicción automatizada y exposición de resultados vía servicio REST

14. DIFICULTADES ENCONTRADAS Y SOLUCIONES APLICADAS

Durante el desarrollo del proyecto surgieron diversos desafíos técnicos que requirieron la aplicación de soluciones específicas.

Uno de los primeros desafíos estuvo relacionado con la adaptación de un proyecto originalmente orientado al análisis de cartas Pokémon hacia un nuevo dominio basado en contenido de Netflix. Este proceso implicó la modificación de estructuras de datos, nombres de campos, procesos ETL y visualizaciones.

Otra dificultad importante surgió durante la integración de las distintas fuentes de información. Debido a que cada fuente presentaba estructuras diferentes, fue necesario establecer criterios de integración basados en el campo *title*, permitiendo relacionar correctamente los registros provenientes del CSV, SQLite y TVMaze.

Asimismo, se identificaron problemas asociados a valores nulos presentes en algunos campos del dataset original. Para resolver esta situación se implementaron procedimientos de limpieza y sustitución de datos faltantes.

Durante la incorporación de la API TVMaze también surgieron desafíos relacionados con la disponibilidad de información para determinados títulos. En algunos casos la API no retornó resultados completos o no encontró coincidencias exactas, generando valores nulos que debieron ser tratados adecuadamente.

En la etapa de despliegue se presentaron dificultades asociadas a la configuración inicial de Docker y Docker Compose. Estas situaciones fueron resueltas mediante la validación de archivos de configuración, corrección de errores de sintaxis y pruebas de ejecución del contenedor.

Finalmente, se realizaron ajustes adicionales en las visualizaciones para asegurar que los gráficos representaran correctamente la información disponible y resultaran comprensibles para los usuarios finales.

La resolución de estos desafíos permitió fortalecer la estabilidad y calidad general de la solución implementada.

15. CONCLUSIONES

El desarrollo del proyecto Netflix Analytics Dashboard permitió aplicar de manera práctica conceptos fundamentales de Ingeniería de Datos mediante la construcción de una solución completa orientada a la integración, procesamiento y visualización de información.

La implementación del proceso ETL demostró la capacidad de consolidar información proveniente de múltiples fuentes heterogéneas, transformarla adecuadamente y ponerla a disposición de herramientas analíticas.

La utilización conjunta de un archivo CSV, una base de datos SQLite y una API externa permitió evidenciar la importancia de la integración de datos dentro de entornos modernos de análisis de información.

Asimismo, el dashboard desarrollado mediante Dash y Plotly demostró cómo los datos procesados pueden transformarse en información visual fácilmente interpretable por los usuarios.

La incorporación de Docker y Docker Compose permitió complementar la solución mediante mecanismos modernos de despliegue y portabilidad, alineándose con prácticas ampliamente utilizadas en entornos profesionales.

Desde una perspectiva académica, el proyecto permitió demostrar competencias relacionadas con integración de datos, calidad de información, automatización de procesos ETL, visualización analítica, documentación técnica y despliegue de aplicaciones.

En consecuencia, puede concluirse que los objetivos definidos inicialmente fueron alcanzados satisfactoriamente y que la solución desarrollada constituye una implementación representativa de los principios fundamentales de la Ingeniería de Datos.

16. RECOMENDACIONES FUTURAS

Aunque la solución desarrollada cumplió adecuadamente con los objetivos planteados, existen diversas oportunidades de mejora que podrían incrementar sus capacidades analíticas y operativas.

Una primera recomendación consiste en incorporar fuentes de datos adicionales que permitan enriquecer aún más la información disponible. Por ejemplo, podrían integrarse APIs relacionadas con calificaciones de usuarios, tendencias de consumo o información financiera asociada a producciones audiovisuales.

También sería posible reemplazar SQLite por motores de bases de datos más robustos, tales como PostgreSQL o MySQL, permitiendo soportar mayores volúmenes de información.

Otra mejora relevante consistiría en automatizar completamente la ejecución periódica del proceso ETL mediante herramientas de orquestación como Apache Airflow o cron jobs.

Desde la perspectiva de visualización, podrían incorporarse filtros dinámicos, paneles personalizados y funcionalidades de exploración avanzada orientadas a distintos perfiles de usuario.

Finalmente, la solución podría evolucionar hacia una arquitectura basada en servicios o infraestructura en la nube, permitiendo una mayor escalabilidad y disponibilidad.

17. REFERENCIAS

Netflix Dataset.

Netflix Titles Dataset. Kaggle. Recuperado desde:

<https://www.kaggle.com/>

TVMaze API.

TVMaze API Documentation. Recuperado desde:

<https://www.tvmaze.com/api>

Python Software Foundation.

Python Documentation. Recuperado desde:

<https://docs.python.org/>

Pandas Development Team.

Pandas Documentation. Recuperado desde:

<https://pandas.pydata.org/>

Plotly Technologies Inc.

Plotly Documentation. Recuperado desde:

<https://plotly.com/>

Plotly Dash Documentation.

Dash Documentation. Recuperado desde:

<https://dash.plotly.com/>

Docker Inc.

Docker Documentation. Recuperado desde:

<https://docs.docker.com/>

SQLite Documentation.

SQLite Documentation. Recuperado desde:

<https://www.sqlite.org/>

Scikit-learn Development Team.

Scikit-learn Documentation. Recuperado desde:

<https://scikit-learn.org/>

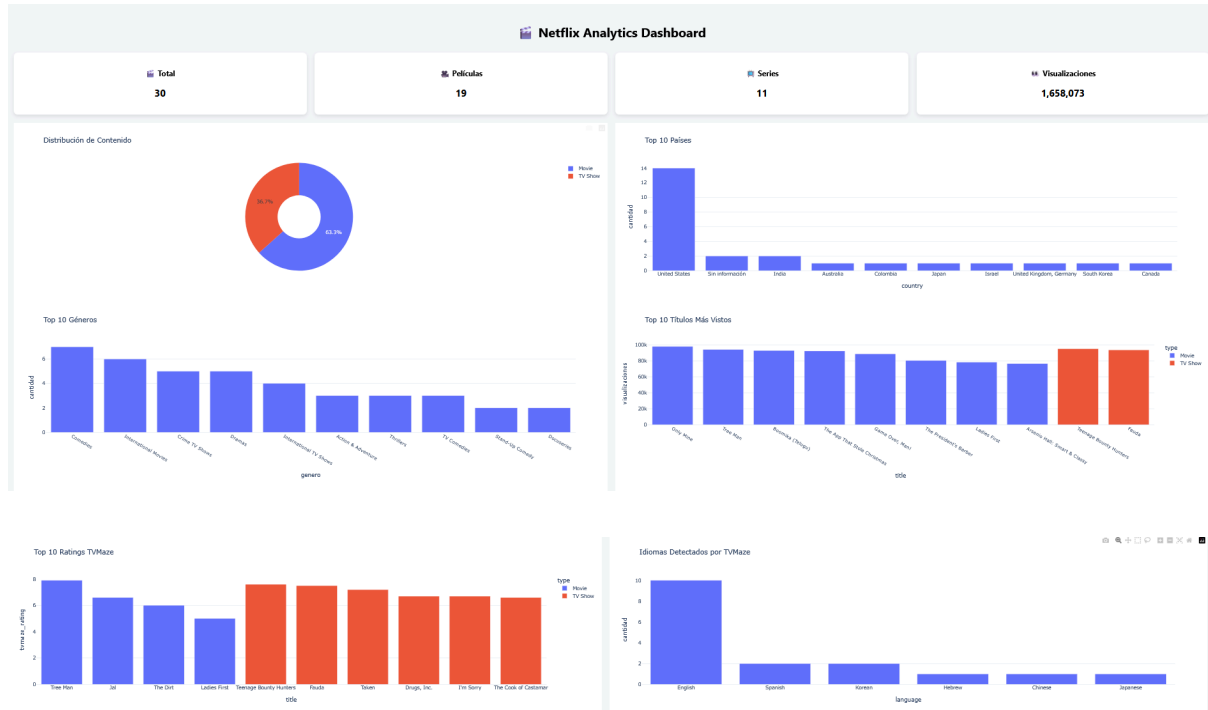
Pallets Projects.

Flask Documentation. Recuperado desde:

<https://flask.palletsprojects.com/>

18. ANEXOS

Anexo A. Dashboard Operativo



Anexo B. Ejecución del ETL

```

Iniciando Pipeline ETL Netflix...

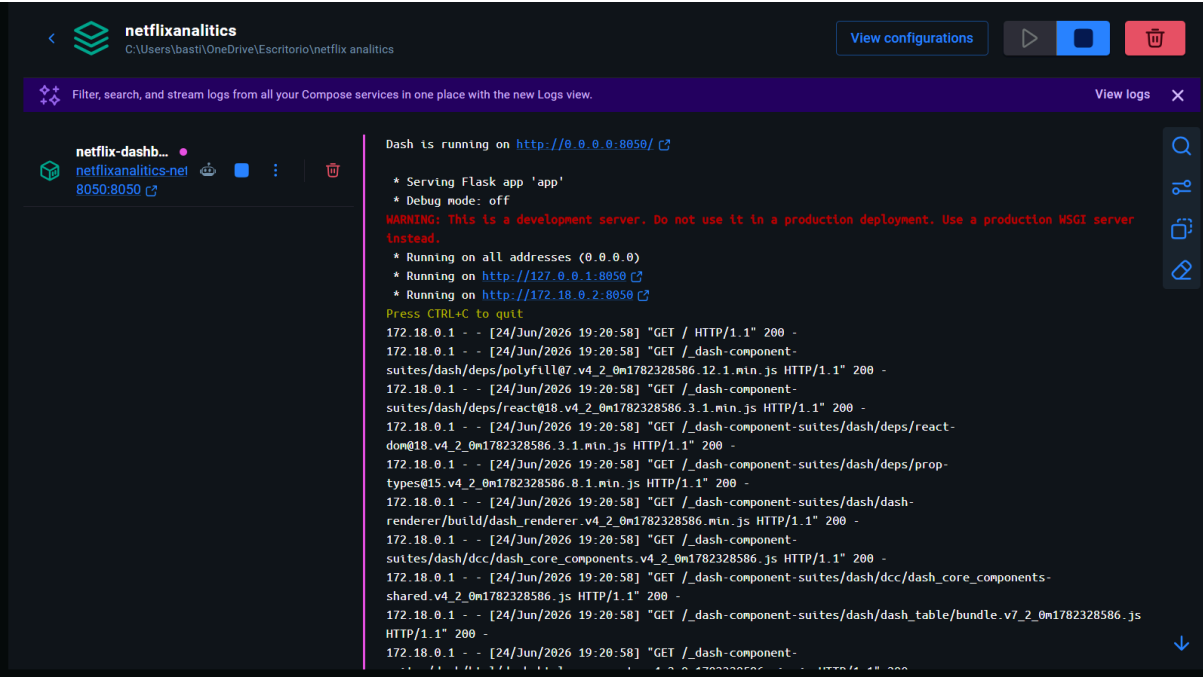
Extrayendo historial de visualizaciones...
Extrayendo catálogo Netflix...
Uniendo visualizaciones con catálogo Netflix...
Consultando TVMaze API...
Guardando dataset final para dashboard...

ETL FINALIZADO CORRECTAMENTE

  title      type  visualizaciones  tvmaze_rating  tvmaze_language  tvmaze_status
0  Game Over, Man!  Movie          88874          NaN          NaN          NaN
1  Arsenio Hall: Smart & Classy  Movie          76648          NaN          NaN          NaN
2  Kazoops!  TV Show           5808          NaN          English        Ended
3  We Are the Champions  TV Show          42664          NaN          English        Ended
4  Pablo Escobar, el patrón del mal  TV Show           5601          NaN          Spanish        Ended
  
```

explicacion luego de los nan que tiene que ver con maze

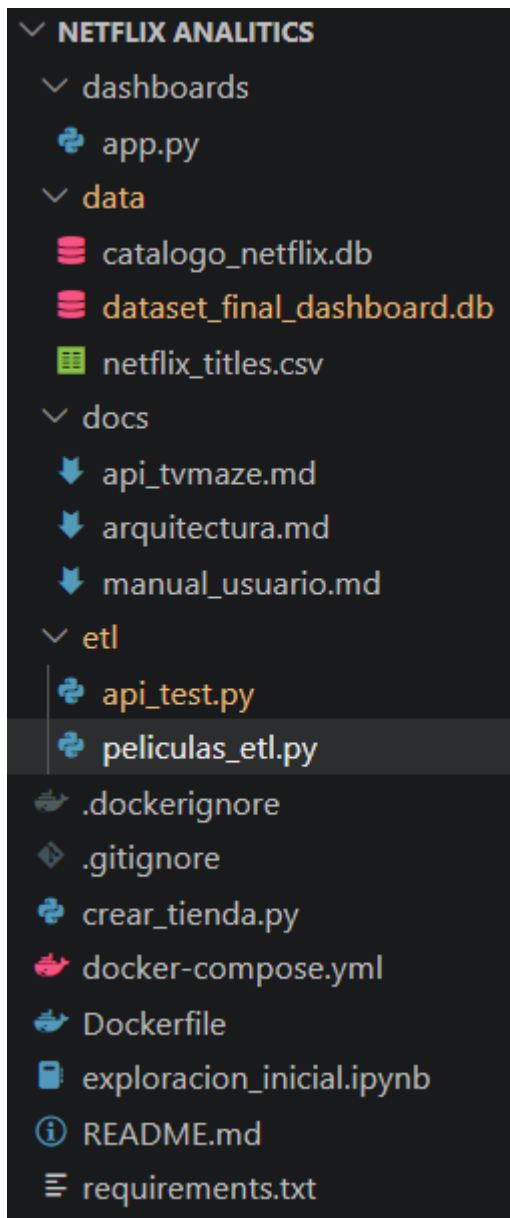
Anexo C. Contenedor Docker



Anexo D. Comando docker ps

C:\Users\basti\OneDrive\Escritorio\netflix_analitics>docker ps							
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES	
f44ab12ada90	netflixanalitics-netflix-dashboard	"python dashboards/a..."	2 minutes ago	Up 2 minutes	0.0.0.0:8050->8050/tcp, [::]:8050->8050/tcp	netflix-analytics	

Anexo E. Estructura del Proyecto



Anexo F. Documentación Técnica

```

api_tvmaze.md X
docs > api_tvmaze.md > abc # Integración API TVMaze > abc ## Beneficio
1  # Integración API TVMaze
2
3  ## Descripción
4
5  TVMaze es una API pública utilizada para obtener información adicional sobre series y programas de televisión.
6
7  Sitio oficial:
8  Follow link (ctrl + click)
9  https://api.tvmaze.com
10
11 ## Objetivo
12
13 Complementar la información existente del catálogo Netflix con datos externos obtenidos en tiempo real.
14
15 ## Endpoint utilizado
16
17 Ejemplo:
18
19 ```text
20 https://api.tvmaze.com/search/shows?q=Wednesday
21 ```
22
23 ## Datos obtenidos
24
25 La API entrega información como:
26
27 * Nombre del programa
28 * Idioma
29 * Estado de emisión
30 * Rating promedio
31
32 ## Campos incorporados al proyecto
33
34 * tvmaze_rating
35 * tvmaze_language
36 * tvmaze_status
37
38 ## Beneficio
39
40 Permite enriquecer el análisis incorporando información externa no disponible en el dataset original.
41

```

```

arquitectura.md X
docs > arquitectura.md > abc # Arquitectura del Sistema > abc ## Base de Datos Final
1  # Arquitectura del Sistema
2
3  ## Descripción General
4
5  Netflix Analytics Dashboard es una solución de análisis de datos que integra información desde múltiples fuentes para generar visualizaciones interactivas orientadas
6  a la toma de decisiones.

```

por cuestiones de espacio solo esta esta parte pero dentro del proyecto esta completa

```

manual_usuario.md X
docs > manual_usuario.md > # Manual de Usuario > ## Ejecución Dashboard
1  # Manual de Usuario
2
3  ## Objetivo
4
5  Visualizar información relevante del catálogo Netflix mediante dashboards interactivos.
6
7  ## Requisitos
8
9  • Python 3.10 o superior
10 • Dependencias instaladas
11 • Dataset disponible
12
13 ## Ejecución ETL
14
15 Desde la raíz del proyecto ejecutar:
16
17 ```bash
18 python etl/peliculas_etl.py
19 ```
20
21 Este proceso:
22
23 1. Lee el catálogo Netflix.
24 2. Lee las visualizaciones desde SQLite.
25 3. Consulta TVMaze.
26 4. Genera la base final para análisis.
27
28 ## Ejecución Dashboard
29
30 Ejecutar:
31
32 ```bash
33 python dashboards/app.py
34 ```
35
36 Abrir navegador:
37
38 http://localhost:8050
39
40
41 ## Indicadores disponibles
42
43 ### KPI
44
45 • Total títulos
46 • Total películas
47 • Total series
48 • Total visualizaciones
49
50 ### Gráficos
51
52 • Distribución de contenido
53 • Top países
54 • Top géneros
55 • Top títulos más vistos
56 • Ratings TVMaze
57 • Idiomas TVMaze
58
59 ## Cierre
60
61 El dashboard permite explorar el catálogo y visualizar tendencias relevantes de forma interactiva.
62

```